

EnerGuard Hackathon · Guida tecnica per i partecipanti

Manutenzione predittiva con supervisione umana per asset critici di rete. Una giornata, 8 ore, 4 tier. Alla fine dovete dimostrare una cosa sola: che un essere umano può capire, monitorare, correggere e fermare il sistema che avete costruito.

0. Cosa ricevete nello starter kit

File	Cosa contiene	Cosa dovete farci
<code>energuard_dataset.csv</code>	2.400 asset di rete con 11 feature e target <code>guasto_entro_30gg</code>	Esplorarlo, addestrarci il modello, interrogarlo criticamente
<code>train_baseline.py</code>	Baseline RandomForest + metriche disaggregate	Eseguirlo subito: è il vostro punto di partenza del Tier 1
<code>oversight_manager.py</code>	Routing HIC/HITL/HOTL, coda di approvazione, emergency stop	Completare i TODO, giustificare le soglie
<code>bias_detector.py</code>	Metriche per sottogruppo + generazione allerte	Estendere con le metriche che ritenete rilevanti
<code>audit_logger.py</code>	Log append-only con catena di hash	Collegarlo a ogni transizione di stato
<code>app.py</code>	Scheletro Streamlit con le 4 sezioni obbligatorie	Costruirci sopra la vostra dashboard

Setup: Python 3.11, `pip install -r requirements.txt` (scikit-learn, pandas, numpy, streamlit, shap, joblib, requests). Ambiente containerizzato già pronto sulle macchine fornite.

Il dataset in dettaglio

Colonna	Tipo	Note
<code>asset_id</code>	str	Identificativo univoco (AST-00001...)
<code>tipo_asset</code>	cat	trasformatore, turbina_eolica, linea_AT, cabina_primaria
<code>area_geografica</code>	cat	Nord, Centro, Sud, Isole. Attenzione: possibile variabile proxy
<code>eta_anni</code>	int	Età del componente
<code>temperatura_media</code>	float	°C, hotspot di esercizio

Colonna	Tipo	Note
<code>vibrazione_indice</code>	float	Indice 0-12, da sensori
<code>carico_pct</code>	float	Carico medio %
<code>umidita_media</code>	float	% ambientale
<code>manutenzioni_ultimi_5anni</code>	int	Interventi registrati
<code>giorni_da_ultima_manutenzione</code>	int	
<code>criticita_utenza</code>	cat	standard, alta, critica (ospedali, infrastrutture essenziali)
<code>guasto_entro_30gg</code>	0/1	Target. Sono i guasti registrati nei sistemi storici

Nota di data governance che vale punti: il target è ciò che è stato *registrato*, non necessariamente ciò che è *accaduto*. In un'azienda reale i processi di segnalazione guasti non sono omogenei tra territori. Trattate il dataset con il sospetto che riservereste a dati storici veri.

Tier 1 · Fondamenta (2 ore) — "Il modello e i suoi limiti"

Obiettivo: baseline funzionante + prova che sapete guardare oltre l'accuratezza media.

Passi operativi: 1. Eseguite `train_baseline.py`. Con i parametri di default ottenete AUC ~0.85. Non spendete la giornata a portarlo a 0.90: da qui in poi i punti si vincono sull'oversight. 2. Scegliete e **giustificate** la soglia decisionale. Domanda guida: in questo dominio costa di più un falso positivo (ispezione inutile) o un falso negativo (guasto non previsto su una cabina che serve un ospedale)? La soglia deve riflettere questa asimmetria. 3. Producete le metriche disaggregate per `area_geografica` e `tipo_asset` (il BiasDetector le calcola). Annotate ogni gap superiore a 15 punti di recall. 4. Esercizio investigativo: confrontate i profili di Sud e Isole (età media, manutenzioni, giorni dall'ultima manutenzione) con i rispettivi tassi di guasto registrati. Se due popolazioni con condizioni simili mostrano tassi molto diversi, formulate almeno due ipotesi e scrivetele: vi serviranno per la relazione di rischio.

Deliverable Tier 1: `predizioni.csv` + mezzo foglio con metriche disaggregate e le anomalie osservate.

Errore che fa perdere tempo: fare feature engineering spinto. Il modello serve solo come "motore" credibile per la dashboard.

Tier 2 · Integrazione (3 ore) — "Costruire l'oversight"

Obiettivo: flusso di approvazione umana reale, end-to-end, non simbolico.

2a. Matrice di routing (45 min)

Compilate una tabella come questa e implementatela in `OversightManager.route()`:

Condizione	Livello	Perché
Utenza critica coinvolta, o riduzione carico su utenza alta/critica	HIC	Interruzione di fornitura a ospedali: veto umano assoluto
$P(\text{guasto}) \geq 0.60$ oppure confidenza < 0.80	HITL	Rischio alto o modello incerto: serve giudizio umano prima di agire
Rischio basso + confidenza alta + azione leggera (routine)	HOTL	Il sistema agisce, l'umano monitora e può intervenire ex post

Le soglie qui sopra sono un default: cambiatele, ma ogni cambiamento va motivato nella Dichiarazione di Oversight. La giuria non chiede "quali soglie avete usato", chiede "perché proprio quelle".

2b. Coda di approvazione (75 min)

Requisiti minimi verificabili: - Ogni decisione HITL/HIC resta in stato `IN_ATTESA` e **nessun codice la esegue** finché un umano non la chiude. Esiste un solo punto di esecuzione nel codice (`_esegui`), e la giuria vi chiederà di dimostrarlo. - Il revisore può: approvare, modificare l'azione e approvare, rifiutare, fare escalation. - La motivazione è obbligatoria e il sistema rifiuta motivazioni sotto i 15 caratteri. Bonus: rilevare il rubber-stamping (motivazioni fotocopia, tempi di revisione sotto i 10 secondi). - Le decisioni HITL non revisionate entro lo SLA vanno in `ESCALATION`, mai eseguite in silenzio.

2c. Emergency stop (30 min)

Deve poter fermare il sistema a tre granularità: globale, per area, per tipo di asset. "Fermare" significa: le nuove raccomandazioni nell'ambito vengono bloccate E quelle già in coda diventano non eseguibili. Anche attivazione e disattivazione dello stop richiedono motivazione e finiscono nell'audit trail.

2d. Prima versione della dashboard (30 min)

Collegate tutto in `app.py`. A fine Tier 2 dovete poter fare una demo di 60 secondi: arriva una raccomandazione ad alto rischio, resta in coda, un operatore la rifiuta con motivazione, il log la registra.

Deliverable Tier 2: dashboard con flusso approva/modifica/rifiuta funzionante + emergency stop dimostrabile.

Tier 3 · Ottimizzazione (2 ore) — "Spiegabilità, bias, monitoraggio"

3a. Spiegabilità (45 min)

Ogni card di decisione deve mostrare i 3 fattori principali che guidano la raccomandazione, **in linguaggio da control room**, non da data scientist.

Male: `SHAP value vibrazione_indice = +0.41` Bene: "Vibrazione a 8.2, molto sopra la norma per un trasformatore di quest'età: è il fattore che pesa di più. Contribuiscono anche 380 giorni senza manutenzione e temperatura di esercizio elevata."

Come farlo, in pratica. Lo starter kit contiene `explainer.py`, già collegato alla dashboard, con due motori intercambiabili:

- **Template** (default, nessuna API): `estrai_fattori()` calcola i tre fattori principali con `shap.TreeExplainer` e `SpiegatoreTemplate` li compone in una frase operativa. Deterministico, gratuito, sempre disponibile.
- **LLM esterno** (opzionale): `SpiegatoreLLM` invia i numeri già calcolati a un modello linguistico via API e ne riceve la traduzione in italiano operativo. Supporta OpenAI, Anthropic, Azure OpenAI e qualunque endpoint compatibile (Ollama, vLLM, LM Studio). Si attiva copiando `.env.example` in `.env` e compilando provider, chiave e modello; `python test_llm.py` verifica il collegamento.

Regola non negoziabile: l'LLM traduce, non decide e non calcola. Riceve la probabilità, la confidenza e i fattori prodotti dal modello predittivo; non ricalcola nulla e non può cambiare la raccomandazione. Il modulo applica tre guardrail — se la risposta cita numeri assenti dai dati, usa gergo tecnico o non menziona il fattore principale, viene scartata e si torna al template — e in caso di errore di rete, quota o timeout il fallback è automatico. Ogni spiegazione dichiara la propria fonte (`template` , `llm:provider/modello` , `template(fallback:...)`) e la chiamata finisce nell'audit trail.

La scelta va motivata in model card. Usare un LLM migliora la leggibilità ma introduce non determinismo, latenza, costo e dipendenza da un fornitore esterno: in un sistema ad alto rischio è una decisione di progettazione, non un dettaglio implementativo. Un team che usa il template e spiega perché vale quanto un team che usa l'LLM e ne governa i rischi. Un team che collega un LLM senza guardrail né fallback perde punti su spiegabilità e affidabilità.

Mostrate sempre anche l'incertezza ("il modello è incerto su questo caso") per contrastare l'automation bias: l'operatore non deve mai vedere solo "l'AI dice sì".

3b. Pannello bias & drift (45 min)

- Tabella metriche per sottogruppo con allerte automatiche sopra soglia (il BiasDetector le genera già; scegliete voi le soglie e difendetele).
- Calibrazione per gruppo: probabilità media predetta vs tasso osservato. Se divergono in un solo gruppo, avete un indizio forte di label bias storico.
- Drift simulato: dividete il test set in "settimane" e mostrate l'andamento di accuratezza e confidenza media nel tempo.
- Tasso di override umano per area: se gli operatori correggono l'AI sistematicamente più spesso in una zona, la dashboard deve farlo notare.

3c. Alert e audit (30 min)

- Soglie che promuovono automaticamente una decisione a un livello di supervisione superiore (es. gap di recall oltre soglia in un'area → tutte le decisioni HOTL di quell'area diventano HITL finché il gap non rientra).
- Sezione audit: log filtrabile + indicatore di integrità della catena di hash sempre visibile.

Deliverable Tier 3: dashboard completa nelle 4 sezioni + almeno un alert automatico funzionante.

Tier 4 · Presentazione (1 ora)

Demo di 5-7 minuti a team. Scaletta consigliata: 1 minuto di scenario, 3 minuti di demo live seguendo il percorso di una decisione (dalla predizione all'audit trail), 2 minuti su bias trovati e limiti dichiarati. Poi domande della giuria, che eseguirà dal vivo i 6 test di verifica descritti nel documento "Indicatori di qualità della dashboard": leggetelo prima di costruire, non dopo.

Documenti da consegnare

1. **Model card** (1 pagina): scopo, dati, metriche globali e per sottogruppo, definizione di confidenza adottata, limiti noti, classificazione di rischio AI Act con motivazione.
 2. **Relazione di impatto** (mezza pagina): rischi identificati, mitigazioni implementate, e soprattutto cosa NON avete risolto. L'onestà sui limiti vale il 10% del punteggio: un limite dichiarato è un punto guadagnato, un limite scoperto dalla giuria è un punto perso due volte.
 3. **Dichiarazione di oversight**: la matrice di routing con le giustificazioni.
-

Riferimenti normativi minimi (quello che vi serve davvero)

- I sistemi AI come componenti di sicurezza nella gestione di infrastrutture critiche (reti elettriche incluse) rientrano nell'Annex III dell'AI Act: alto rischio. Requisiti pieni dal 2 agosto 2026, cioè tra pochi giorni rispetto a oggi.
- Art. 14: le persone che supervisionano devono poter comprendere capacità e limiti del sistema, monitorarne il funzionamento, restare consapevoli dell'automation bias, interpretare correttamente l'output, decidere di non usarlo o ignorarlo, intervenire o interrompere il sistema. Ogni sezione della vostra dashboard risponde a uno di questi punti: se una sezione non risponde a nessuno, è decorazione.
- Art. 13: trasparenza verso il deployer (finalità, accuratezza attesa, rischi noti). La model card è la vostra risposta.
- Art. 10: qualità e governance dei dati, incluso l'esame di possibili bias. La vostra indagine sul dataset è la risposta.
- Art. 12: registrazione automatica degli eventi. L'audit trail è la risposta.

Domanda finale a cui prepararvi: nel mondo reale, chi sarebbe il provider e chi il deployer del vostro sistema, e quali obblighi ricadrebbero su ciascuno?